

Introduction to LangGraph

Overview

The project was designed to simulate a workflow using LangGraph, which is based on an airport security screening scenario. The purpose of this assignment is to learn how LangGraph can be applied for workflow, routing, and state transitions using a graph-based execution model. The simulation is based on how passengers pass through different security screening procedures before reaching the departure gate.

Objective

- Create a workflow with LangGraph
- Create nodes and edges
- Implement conditional routing
- Simulate different paths
- Visualize the workflow graph
- Run the graph multiple times with different results

Methodology

State Definition

A state was defined for tracking passenger flow through the system.

The state also helps determine which way the passenger will go in the workflow based on their type.

Nodes

The following nodes were added to the graph:

- start_node – The passenger arrives at security
- tsa_screening – The passenger goes through TSA PreCheck
- regular_screening – The passenger goes through regular screening
- additional_screening – The passenger goes through additional security
- gates – The passenger goes to the gates
- END – The process is complete

Each node is a step in the workflow for airport security.

Conditional Routing

Two decision functions were added:

1. Passenger type
 - 20% - TSA PreCheck
 - 80% - Regular Screening
2. Additional screening
 - 10% - Additional screening
 - 90% - Cleared for gates

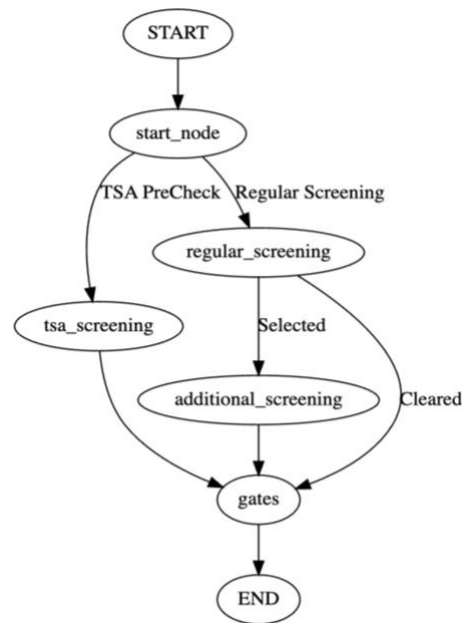
The edges are decision-making processes.

Graph Workflow

The workflow for the graph is defined as follows:

```
"START" -> start_node -> Passenger Type Decision
          -> TSA Screening -> Gates -> END
          -> Regular Screening -> Additional Screening -> Gates -> END
          -> Regular Screening -> Gates -> END
```

This is a sample graph that shows how branching, conditional routing, and path merge are implemented.



Results

To execute the graph, the code invokes the function using the following statement: `graph.invoke({})`

The graph has been executed multiple times, and different paths have been created based on the probabilities used in the code.

Example output:

"Passenger" -> TSA PreCheck -> Gates -> End

"Passenger" -> Regular Screening -> Gates -> End

"Passenger" -> Regular Screening -> Additional Screening -> Gates -> End

This is a sample output that shows how LangGraph is used for handling conditional routing and multiple paths in a workflow.

Path	Description
TSA → Gates	Passenger uses TSA PreCheck
Regular → Gates	Passenger cleared after regular screening
Regular → Additional → Gates	Passenger selected for additional screening

6. Key Concepts Learned

The key concepts that were learned from this assignment are as follows:

- State Graph Structure
- Nodes and Edges
- Conditional Edges
- Workflow Modeling using Graphs
- Simulation of Decision-Based Processes
- Graph Visualization
- Execution of Flows using `graph.invoke()`

7. Conclusion

This project was about how LangGraph could be used for modeling a workflow system with certain conditions and probabilistic decision processes. The simulation of decision-based processes, such as airport security checks, showed how a passenger could pass through different paths depending on certain conditions and probabilities. LangGraph is a systematic way of designing and executing a workflow system.